>_ MCMP113 · Week 6

Loops: the `for` Loop

Counting, accumulating, nested loops, `break` and `continue`

Asst. Prof. Hasan Oğuz, PhD
Introduction to Basic Algorithms and Programming · Istanbul Okan University
Vocational School · 2026–2027 Fall

How to use these notes

These notes follow the Week 6 slides and add what does not fit on a slide: the exact rules of the `for` statement, flowcharts, trace tables, all thirteen example programs with real sample runs, the common mistakes, a glossary and graded exercises. Every listing is the tested program from the `code` folder (`06_01` ... `06_13`); open it in VS Code and run it while you read. Typed input in the sample runs is shown in **amber**. Work through Sections 3–7 before the lab, then do the exercises in Section 10. `while` and `do-while` follow in Week 7; arrays in Week 9.

Contents

1 Learning outcomes **2**

2 Why loops? **2**

3 Anatomy of the `for` statement **3**
3.1 Execution order . . . 3
3.2 Flowchart and pseudocode . . . 3
3.3 Trace tables . . . 4

4 Counting patterns **4**
4.1 Up, down and in steps . . . 4
4.2 How many passes? . . . 5
4.3 Scope of the loop variable . . . 6

5 Accumulators and counters **6**
5.1 Worked example: sum, average and pass count . . . 7
5.2 Product accumulator: factorial and overflow . . . 8
5.3 Using the loop variable in a calculation 9
5.4 Compound interest, year by year . . 10
5.5 Minimum and maximum without arrays . . . 10
5.6 Fibonacci numbers: carrying two values . . . 11

6 Nested loops **12**
6.1 Multiplication grid . . . 12
6.2 Star patterns . . . 13
6.3 Check yourself: predict the output . 14

7 `break` and `continue` **15**
7.1 Prime test by trial division . . . 16
7.2 All primes up to N . . . 18
7.3 FizzBuzz . . . 19

8 Common mistakes **20**

9 Summary **20**

10 Exercises **21**
10.1 In-class exercises (pairs) . . . 21
10.2 Homework . . . 22
10.3 Extra practice . . . 22

References and further reading **23**

>_ 1 Learning outcomes

After this week you should be able to:

1. name the four parts of a `for` loop (initialization, condition, update, body) and state the order in which they run;
2. draw the flowchart of a `for` loop and trace a loop by hand in a trace table;
3. predict how many times a loop body runs, including nested loops;
4. use accumulators and counters to compute sums, products, counts, averages, minima and maxima;
5. write nested loops for tables and character patterns;
6. use `break` and `continue` correctly;
7. recognise and fix the classic loop mistakes: off-by-one, stray semicolon, changing the loop variable, infinite loops, overflow.

>_ 2 Why loops?

Week 5 gave us **selection**: a decision that is taken *once*. The third control structure of Böhm and Jacopini (Week 1) is **repetition**: executing a block of statements again and again. Suppose we must print the numbers 1 to 100. With what we know so far we would write one hundred almost identical lines (left). This is tedious, error-prone, and, most importantly, impossible when the number of lines is only known at run time ("print 1 to n, where the user types n"). A loop writes the repeated statement once (right):

```
Console.WriteLine(1);
Console.WriteLine(2);
Console.WriteLine(3);
// ... 96 more almost identical lines ...
Console.WriteLine(100);
```

```
for (int i = 1; i <= 100; i++)
{
    Console.WriteLine(i);
}
```

In Week 1 we already met repetition in pseudocode (Algorithm 3, sum $1 + 2 + \cdots + N$) with a **WHILE** loop and a counter i. The counter had three jobs: start ($i \leftarrow 1$), test ($i \leq N$), step ($i \leftarrow i + 1$). The **FOR** form collects the three jobs in one line:

Week 1: sum $1..N$ with WHILE

```
START
READ N
sum ← 0
i ← 1
WHILE i ≤ N
    sum ← sum + i
    i ← i + 1
END WHILE
PRINT sum
END
```

The same algorithm with FOR

```
START
READ N
sum ← 0
FOR i ← 1 TO N
    sum ← sum + i
END FOR
PRINT sum
END
```

A `for` loop is the natural choice when the *number of repetitions is known before the loop starts* (a **counter-controlled** loop). When the loop must run "until something happens" (until the user types 0, until the guess is right) we will prefer `while` (Week 7).

>_ 3 Anatomy of the `for` statement

Syntax – the for statement

```
for (initialization; condition; update)
{
body
}
```

Part	Example	Meaning
① **Initialization**	`int i = 1`	Runs *once*, before anything else. Usually declares the **loop variable** (the counter).
② **Condition**	`i <= 5`	A `bool` expression tested *before every pass*. `true`: run the body; `false`: leave the loop.
③ **Body**	`{ … }`	The statements that are repeated. One execution of the body is called a **pass** or **iteration**.
④ **Update**	`i++`	Runs *after every pass*; then the condition is tested again.

The two semicolons inside the parentheses separate the three header parts; there is *no* semicolon after the closing parenthesis (Section 8). The update can be any statement that changes the counter: `i++`, `i--`, `i += 2`, `i *= 2`.

3.1 Execution order

The order is always ① → ② → ③ → ④ → ② → ③ → ④ → · · · → ② (false) → exit. Table 1 executes `for (int i = 1; i <= 3; i++) Console.Write(i);` step by step; it prints `123`.

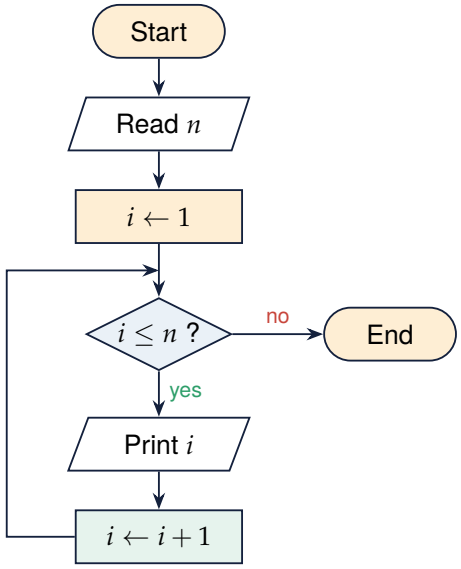
Step	Part	What happens	Output so far
1	① init	`i = 1`	
2	② test	`1 <= 3` → true	
3	③ body	`Write(1)`	`1`
4	④ update	`i++` → `i = 2`	`1`
5–7	②③④	`2 <= 3` true, `Write(2)`, `i = 3`	`12`
8–10	②③④	`3 <= 3` true, `Write(3)`, `i = 4`	`123`
11	② test	`4 <= 3` → false: exit	`123`

Table 1: Execution order of a loop with three passes.

Count the steps: the initialization runs once, the body three times, the update three times, and the condition *four* times (three times true, once false). In general a loop that makes n passes tests its condition $n + 1$ times. Because the condition is tested *before* the body, `for` is a **pre-test loop**: if the condition is false at the start, the body runs zero times.

3.2 Flowchart and pseudocode

There is no special flowchart symbol for `for`: it is built from a process box (initialization), a decision (condition), the body, a second process box (update) and a **back-edge** that returns to the decision (Figure 1). The back-edge joins the flow *above* the decision.



Algorithm 1 – Print $1..n$

```
1 START
2 READ n
3 FOR i ← 1 TO n
4     PRINT i
5 END FOR
6 END
```

```
int n = int.Parse(Console.ReadLine());
for (int i = 1; i <= n; i++)
{
    Console.WriteLine(i);
}
```

Figure 1: The `for` loop as a flowchart, in pseudocode and in C#.

Pseudocode convention. **FOR** $i \leftarrow a$ **TO** b means that i takes the values $a, a+1, \ldots, b$ (step $+1$, both ends included); this is `for (int i = a; i <= b; i++)`. Other steps are written with **STEP**: **FOR** $i \leftarrow 10$ **TO** 1 **STEP** -1 is `for (int i = 10; i >= 1; i--)`.

3.3 Trace tables

A trace table executes a loop by hand: one row per pass, one column per variable, plus a start row and an exit row. For the fragment on the left we obtain the table on the right.

```
int sum = 0;
for (int i = 1; i <= 5; i++)
{
    sum += i;
}
Console.WriteLine(sum);
```

Pass	i (before test)	`i <= 5` ?	*sum* (after body)
start	1	–	0
1	1	true	1
2	2	true	3
3	3	true	6
4	4	true	10
5	5	true	15
exit	6	false	**15**

The program prints `15`, in agreement with Gauss's formula $\frac{5 \cdot 6}{2} = 15$ from Week 1. The exit row is part of the trace: it shows the value that stopped the loop.

>_ 4 Counting patterns

4.1 Up, down and in steps

Example 1 shows the four basic counting patterns. Note that each loop declares its own `i`; this is legal because each `i` exists only inside its own loop (Section 4.3).

Listing 1: Example 1 – counting patterns (`06_01_CountingPatterns`)

```
// MCMP113 · Week 6 · Example 1 — Counting patterns
// Demonstrates counting up, counting down and counting in steps with a for loop.

Console.Write("Count up to: ");
int n = int.Parse(Console.ReadLine());

// 1) count up: 1, 2, ..., n
Console.Write("Up:     ");
for (int i = 1; i <= n; i++)
{
    Console.Write($"{i} ");
}
Console.WriteLine();

// 2) count down: n, n-1, ..., 1
Console.Write("Down:   ");
for (int i = n; i >= 1; i--)
{
    Console.Write($"{i} ");
}
Console.WriteLine();

// 3) count in steps of 2: 0, 2, 4, ... (never above n)
Console.Write("Even:   ");
for (int i = 0; i <= n; i += 2)
{
    Console.Write($"{i} ");
}
Console.WriteLine();

// 4) count down in steps of 5, starting at 5*n
Console.Write("By -5:  ");
for (int i = 5 * n; i > 0; i -= 5)
{
    Console.Write($"{i} ");
}
Console.WriteLine();
```

Sample run

```
Count up to: 10
Up:     1 2 3 4 5 6 7 8 9 10
Down:   10 9 8 7 6 5 4 3 2 1
Even:   0 2 4 6 8 10
By -5:  50 45 40 35 30 25 20 15 10 5
```

Direction rule

Counting *up*: start low, test with `<` or `<=`, update with `++` or `+=`. Counting *down*: start high, test with `>` or `>=`, update with `--` or `-=`. Mixing the two gives either zero passes or a (practically) endless loop.

4.2 How many passes?

Before running a loop, estimate how often its body runs. For a loop `for (i = a; i <= b; i += s)` with $a \le b$ and $s > 0$ the number of passes is

$$\text{passes} = \left\lfloor \frac{b - a}{s} \right\rfloor + 1$$

(integer division in C#). With `i < b` replace b by $b - 1$. Every row of the next table was checked by running the loop.

Header	Values of i	Passes
`for (int i = 1; i <= 10; i++)`	$1, 2, \ldots, 10$	10
`for (int i = 0; i < 10; i++)`	$0, 1, \ldots, 9$	10
`for (int i = 1; i < 10; i++)`	$1, 2, \ldots, 9$	9
`for (int i = 0; i <= 20; i += 2)`	$0, 2, \ldots, 20$	11
`for (int i = 10; i > 0; i -= 3)`	$10, 7, 4, 1$	4
`for (int i = 1; i <= 100; i *= 2)`	$1, 2, 4, \ldots, 64$	7
`for (int i = 5; i < 5; i++)`	none	0

The last-but-one loop multiplies instead of adding; such a loop runs about $\log_2 n$ times (compare the halving strategy of Week 1). The last loop shows that a `for` body may run zero times.

4.3 Scope of the loop variable

A variable declared in the header exists only from the header to the closing brace of the loop. Using it afterwards does not compile (left). If the final value is really needed, declare the variable before the loop and only *assign* it in the header (right):

```
int sum = 0;
for (int i = 1; i <= 5; i++)
{
    sum += i;
}
Console.WriteLine($"Last i = {i}");
```

```
int sum = 0;
int i;              // declared outside
for (i = 1; i <= 5; i++)
{
    sum += i;
}
Console.WriteLine($"Last i = {i}");
```

Build output of the left program

```
error CS0103: The name 'i' does not exist in the current context
```

The right program prints `Last i = 6`: the first value that made the condition false. For the same reason, an accumulator such as `sum` must be declared *before* the loop if it is to be used after it. Inside nested loops the inner counter needs a different name: declaring a second `int i` inside a loop over `i` is error CS0136.

>_ 5 Accumulators and counters

Most loops compute something from all the values they visit. The pattern always has three steps: **initialise before** the loop, **update inside** the loop, **use after** the loop.

Role	Before the loop	Inside the loop	Why
Sum	`int sum = 0;`	`sum += x;`	0 is neutral for $+$.
Product	`long product = 1;`	`product *= x;`	1 is neutral for $\times$; starting at 0 keeps the result 0.
Counter	`int count = 0;`	`if (x >= 50) count++;`	counts the passes that meet a condition.
Average	(after the loop)	–	`(double)sum / count`, only if `count > 0`.

5.1 Worked example: sum, average and pass count

Problem. Read how many grades there are, then each grade; print the sum, the average and how many grades are at least 50.

Algorithm 2 – Sum, average, pass count

```
START
READ count
sum ← 0; passed ← 0
FOR i ← 1  TO count
    READ grade
    sum ← sum + grade
    IF grade ≥ 50 THEN
        passed ← passed + 1
    END IF
END FOR
IF count > 0 THEN
    PRINT sum, sum/count, passed
ELSE
    PRINT "No grades entered."
END IF
END
```

Test plan (expected values computed by hand):

Input	Expected
5 grades: 72.5, 45, 88, 60, 39.5	sum 305, average $305/5 = 61$, passed 3 (72.5, 88, 60)
0 grades	`No grades entered.`

Listing 2: Example 2 – sum, average and pass count (`06_02_SumAndAverage`)

```
// MCMP113 · Week 6 · Example 2 — Sum, average and count of N grades
// Demonstrates accumulators (sum) and counters (passed) inside a for loop.

Console.Write("How many grades? ");
int count = int.Parse(Console.ReadLine());

double sum = 0;   // accumulator: starts at 0
int passed = 0;   // counter: starts at 0

for (int i = 1; i <= count; i++)
{
    Console.Write($"Grade {i}: ");
    double grade = double.Parse(Console.ReadLine());
    sum += grade;              // add to the running total
    if (grade >= 50)
    {
        passed++;              // count the passing grades
    }
}

if (count > 0)
{
    double average = sum / count;
    Console.WriteLine($"Sum     = {sum:F2}");
    Console.WriteLine($"Average = {average:F2}");
```

```
    Console.WriteLine($"Passed  = {passed} of {count}");
}
else
{
    Console.WriteLine("No grades entered.");
}
```

Sample run

```
How many grades? 5
Grade 1: 72.5
Grade 2: 45
Grade 3: 88
Grade 4: 60
Grade 5: 39.5
Sum     = 305.00
Average = 61.00
Passed  = 3 of 5
```

Sample run (edge case)

```
How many grades? 0
No grades entered.
```

The decimal point in `72.5` is accepted on every PC because of the setting `InvariantGlobalization` in our project template (Week 3). `sum` is a `double`, so `sum / count` is a floating-point division.

5.2 Product accumulator: factorial and overflow

$n! = 1 \cdot 2 \cdot 3 \cdots n$ is a product, so the accumulator starts at 1. The type is `long` because `int` is already too small for $13! = 6\,227\,020\,800$.

Listing 3: Example 3 – factorial table (`06_03_Factorial`)

```
// MCMP113 · Week 6 · Example 3 — Factorial table with long
// Demonstrates a product accumulator (starts at 1) and silent overflow after 20!.

Console.Write("n = ");
int n = int.Parse(Console.ReadLine());

long factorial = 1;   // product: starts at 1, not 0
for (int i = 1; i <= n; i++)
{
    factorial *= i;   // factorial = factorial * i
    Console.WriteLine($"{i,2}! = {factorial,26:N0}");
}
Console.WriteLine($"long.MaxValue = {long.MaxValue:N0}");
```

Sample run

```
n = 22
 1! =                          1
 2! =                          2
 3! =                          6
 4! =                         24
 5! =                        120
 6! =                        720
 7! =                      5,040
 8! =                     40,320
 9! =                    362,880
10! =                  3,628,800
11! =                 39,916,800
12! =                479,001,600
13! =              6,227,020,800
14! =             87,178,291,200
15! =          1,307,674,368,000
16! =         20,922,789,888,000
17! =        355,687,428,096,000
18! =      6,402,373,705,728,000
19! =    121,645,100,408,832,000
20! =  2,432,902,008,176,640,000
21! = -4,249,290,049,419,214,848
22! = -1,250,660,718,674,968,576
long.MaxValue = 9,223,372,036,854,775,807
```

Up to $20! = 2\,432\,902\,008\,176\,640\,000$ every value is correct. The true value $21! = 51\,090\,942\,171\,709\,440\,000 \approx 5.1 \cdot 10^{19}$ exceeds `long.MaxValue` $\approx 9.2 \cdot 10^{18}$. C# does not stop: in the default *unchecked* context the result **overflows** and silently wraps around; the printed value is $21! - 3 \cdot 2^{64}$, which happens to be negative.

Silent overflow

A loop that multiplies grows very fast. Either choose a type that is large enough, or let C# detect the problem: `factorial = checked(factorial * i);` throws an `OverflowException` instead of printing a wrong number (Week 4).

5.3 Using the loop variable in a calculation

Listing 4: Example 4 – multiplication table (`06_04_MultiplicationTable`)

```
// MCMP113 · Week 6 · Example 4 — Multiplication table of one number
// Demonstrates using the loop variable in a calculation and aligned output {x,3}.

Console.Write("Table of: ");
int number = int.Parse(Console.ReadLine());

for (int i = 1; i <= 10; i++)
{
    Console.WriteLine($"{number} x {i,2} = {number * i,3}");
}
```

Sample run

```
Table of: 7
7 x  1 =   7
7 x  2 =  14
7 x  3 =  21
7 x  4 =  28
7 x  5 =  35
7 x  6 =  42
7 x  7 =  49
7 x  8 =  56
7 x  9 =  63
7 x 10 =  70
```

Inside the body the loop variable is an ordinary `int`: we may read it, compute with it and print it. The format items `{i,2}` and `{number * i,3}` pad to 2 and 3 characters so that the columns line up.

What we must *not* do is assign to `i` in the body: the header alone controls the counter (Section 8). To print the tables of 1 to 10 at once, this loop is placed inside another loop (Section 6).

5.4 Compound interest, year by year

The balance is an accumulator that grows by a percentage: each pass computes this year's interest from the *current* balance and adds it. `decimal` is the right type for money (Week 3).

Listing 5: Example 13 – compound interest table (`06_13_CompoundInterest`)

```
// MCMP113 · Week 6 · Example 13 — Compound interest, year by year
// Demonstrates an accumulator that grows by a percentage and a formatted table.

Console.Write("Deposit (TL): ");
decimal balance = decimal.Parse(Console.ReadLine());
Console.Write("Annual interest rate (%): ");
decimal rate = decimal.Parse(Console.ReadLine());
Console.Write("Years: ");
int years = int.Parse(Console.ReadLine());

Console.WriteLine("Year      Interest       Balance");
for (int year = 1; year <= years; year++)
{
    decimal interest = balance * rate / 100;  // this year
    balance += interest;                      // add to balance
    Console.WriteLine($"{year,4} {interest,13:F2} {balance,13:F2}");
}
```

Sample run

```
Deposit (TL): 10000
Annual interest rate (%): 40
Years: 5
Year      Interest       Balance
   1       4000.00      14000.00
   2       5600.00      19600.00
   3       7840.00      27440.00
   4      10976.00      38416.00
   5      15366.40      53782.40
```

Check. With the closed formula $A = P(1+r)^n$: $10\,000 \cdot 1.4^5 = 10\,000 \cdot 5.37824 = 53\,782.40$, exactly the last line of the table. Every year earns more interest than the previous one, because interest is also paid on earlier interest. "How many years until the money doubles?" cannot be answered with a fixed number of passes; that needs a `while` loop (Week 7).

5.5 Minimum and maximum without arrays

To find the smallest and the largest of N numbers we do not need to store the numbers: each one is compared with the "best so far" and then forgotten.

Listing 6: Example 12 – smallest and largest (`06_12_MinMax`)

```
// MCMP113 · Week 6 · Example 12 — Smallest and largest of N numbers (no arrays)
// Demonstrates "best so far" variables initialised with the first input.

```

```
Console.Write("How many numbers? ");
int count = int.Parse(Console.ReadLine());

double min = 0;
double max = 0;
for (int i = 1; i <= count; i++)
{
    Console.Write($"Number {i}: ");
    double x = double.Parse(Console.ReadLine());
    if (i == 1 || x < min)     // first number, or a new minimum
    {
        min = x;
    }
    if (i == 1 || x > max)     // first number, or a new maximum
    {
        max = x;
    }
}

if (count > 0)
{
    Console.WriteLine($"Smallest = {min}");
    Console.WriteLine($"Largest  = {max}");
    Console.WriteLine($"Range    = {max - min}");
}
else
{
    Console.WriteLine("Nothing to compare.");
}
```

Sample run

```
How many numbers? 5
Number 1: 17
Number 2: -4
Number 3: 23.5
Number 4: 8
Number 5: 0
Smallest = -4
Largest  = 23.5
Range    = 27.5
```

i	x	reason	min	max
1	17	first number	17	17
2	-4	$-4 < 17$	-4	17
3	23.5	$23.5 > 17$	-4	23.5
4	8	no change	-4	23.5
5	0	no change	-4	23.5

The condition `i == 1 || x < min` makes the first number both the minimum and the maximum; thanks to short-circuit evaluation (Week 4) the comparison with the not-yet-meaningful `min` is not even evaluated when `i == 1`. Starting with `min = 0` would be wrong: for the inputs $17, 23.5, 8$ the "minimum" would stay 0, a value nobody entered.

5.6 Fibonacci numbers: carrying two values

In the sequence $0, 1, 1, 2, 3, 5, 8, \ldots$ every term is the sum of the previous two: $F_{k+2} = F_k + F_{k+1}$. The loop keeps two variables and shifts them one step per pass.

Listing 7: Example 10 – first n Fibonacci numbers (`06_10_Fibonacci`)

```
// MCMP113 · Week 6 · Example 10 — First N Fibonacci numbers
// Demonstrates carrying two values from one pass to the next.

Console.Write("How many terms? ");
int n = int.Parse(Console.ReadLine());

long previous = 0;   // F(0)
```

```
long current = 1;    // F(1)
for (int i = 1; i <= n; i++)
{
    Console.Write($"{previous} ");
    long next = previous + current;  // sum of the last two
    previous = current;              // shift one step forward
    current = next;
}
Console.WriteLine();
```

Sample run

```
How many terms? 12
0 1 1 2 3 5 8 13 21 34 55 89
```

Trace for $n = 6$ (right): the output `0 1 1 2 3 5` is the first six numbers of the run above.

i	printed	*next*	*previous*	*current*
start			0	1
1	0	1	1	1
2	1	2	1	2
3	1	3	2	3
4	2	5	3	5
5	3	8	5	8
6	5	13	8	13

The order of the three assignments matters: if `previous = current` came first, the old `previous` would be lost. With `long`, F_{92} is the largest Fibonacci number that fits.

>_ 6 Nested loops

A loop may contain another loop. The **inner loop** runs *completely* for every single pass of the **outer loop**, like the minute hand of a clock, which makes a full turn for every step of the hour hand.

```
for (int i = 1; i <= 3; i++)          // outer
{
    for (int j = 1; j <= 2; j++)      // inner
    {
        Console.Write($"({i},{j}) ");
    }
    Console.WriteLine();
}
```

Output

```
(1,1) (1,2)
(2,1) (2,2)
(3,1) (3,2)
```

The inner initialization `j = 1` runs again at the start of every outer pass, so j restarts from 1. The inner body runs $3 \times 2 = 6$ times. In general, an outer loop with n passes around an inner loop with m passes executes the inner body $n \cdot m$ times; if the inner loop runs $j = 1, \ldots, i$ the count is $1 + 2 + \cdots + n = \frac{n(n+1)}{2}$. The `WriteLine()` after the inner loop (but inside the outer loop) ends each row; its position decides the shape of the output.

6.1 Multiplication grid

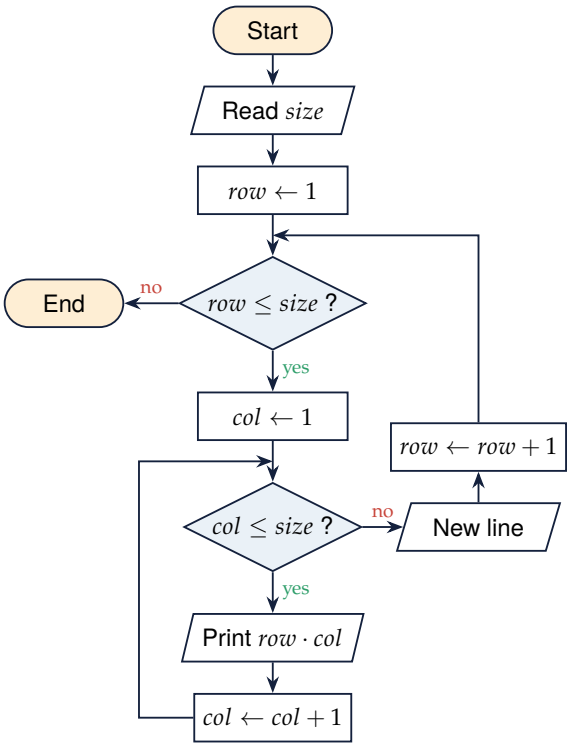
Listing 8: Example 5 – multiplication grid (`06_05_MultiplicationGrid`)

```
// MCMP113 · Week 6 · Example 5 — Multiplication grid (nested loops)
// Demonstrates an outer loop for rows and an inner loop for columns.

Console.Write("Grid size: ");
int size = int.Parse(Console.ReadLine());

for (int row = 1; row <= size; row++)      // outer: rows
{
```

```
    for (int col = 1; col <= size; col++)  // inner: columns
    {
        Console.Write($"{row * col,4}");   // 4 chars wide
    }
    Console.WriteLine();                   // end of row
}
```



Sample run

```
Grid size: 10
   1   2   3   4   5   6   7   8   9  10
   2   4   6   8  10  12  14  16  18  20
   3   6   9  12  15  18  21  24  27  30
   4   8  12  16  20  24  28  32  36  40
   5  10  15  20  25  30  35  40  45  50
   6  12  18  24  30  36  42  48  54  60
   7  14  21  28  35  42  49  56  63  70
   8  16  24  32  40  48  56  64  72  80
   9  18  27  36  45  54  63  72  81  90
  10  20  30  40  50  60  70  80  90 100
```

The grid has two back-edges: the inner one returns to the column test, the outer one to the row test. If `Console.WriteLine()` were moved into the inner loop, every product would be on its own line (100 lines); if it were moved after the outer loop, all 100 products would be on one line.

6.2 Star patterns

For any character pattern, first make a table "row number → how many characters of each kind", find the formula, and only then write the loops.

Listing 9: Example 6 – star patterns (`06_06_StarPatterns`)

```
// MCMP113 · Week 6 · Example 6 — Star patterns
// Demonstrates nested loops whose inner bound depends on the outer variable.

Console.Write("Rows: ");
int rows = int.Parse(Console.ReadLine());

// A) rectangle: every row has the same number of stars
Console.WriteLine("A) Rectangle");
for (int r = 1; r <= rows; r++)
{
    for (int c = 1; c <= 2 * rows; c++)
    {
        Console.Write("*");
    }
    Console.WriteLine();
}

```

```
// B) right triangle: row r has r stars
Console.WriteLine("B) Right triangle");
for (int r = 1; r <= rows; r++)
{
    for (int c = 1; c <= r; c++)
    {
        Console.Write("*");
    }
    Console.WriteLine();
}

// C) pyramid: row r has rows - r spaces, 2r - 1 stars
Console.WriteLine("C) Pyramid");
for (int r = 1; r <= rows; r++)
{
    for (int s = 1; s <= rows - r; s++)
    {
        Console.Write(" ");
    }
    for (int c = 1; c <= 2 * r - 1; c++)
    {
        Console.Write("*");
    }
    Console.WriteLine();
}
```

Sample run

```
Rows: 4
A) Rectangle
********
********
********
********
B) Right triangle
*
**
***
****
C) Pyramid
   *
  ***
 *****
*******
```

In the *rectangle* the inner bound is fixed (`2 * rows`); in the *right triangle* the inner bound is the outer variable `r`, so row r has r stars. The *pyramid* needs two inner loops one after the other (dots stand for spaces):

row r	spaces $= rows - r$	stars $= 2r - 1$	line
1	3	1	`···*`
2	2	3	`··***`
3	1	5	`·*****`
4	0	7	`*******`

6.3 Check yourself: predict the output

Write the output of each fragment on paper, then compare with the answers below. (These are quiz questions Q1–Q3 of the slides.)

Listing 10: Q1

```
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= i; j++)
    {
        Console.Write(j);
    }
    Console.WriteLine();
}
```

Listing 11: Q2

```
for (int i = 3; i >= 1; i--)
{
    for (int j = 1; j <= i; j++)
    {
        Console.Write("#");
    }
    Console.WriteLine();
}
```

Listing 12: Q3

```
for (int i = 1; i <= 4; i++)
{
    for (int j = 1; j <= 4; j++)
    {
        if ((i + j) % 2 == 0)
        {
            Console.Write("#");
        }
        else
        {
            Console.Write(".");
        }
    }
    Console.WriteLine();
}
```

Answers (real program output)

Q1: 1 / 12 / 123 (row i prints $1..i$).

Q2: `###` / `##` / `#` (the outer loop counts down).

Q3: `#.#.` / `.#.#` / `#.#.` / `.#.#` (even $i+j$ gives #: a checkerboard).

Cover this box until you have written your own answers.

>_ 7 `break` and `continue`

Jump statements in loops

`break;` leaves the loop immediately; execution continues with the first statement after the loop, and the update does not run any more.

`continue;` skips the rest of the current pass; in a `for` loop the **update still runs**, then the condition is tested again.

Both act only on the **innermost** loop that contains them.

Listing 13: Example 7 – `break` and `continue` (`06_07_BreakContinue`)

```
1  // MCMP113 · Week 6 · Example 7 — break and continue
2  // Demonstrates skipping one pass (continue) and leaving the loop early (break).
3
4  // Part 1: continue -> skip the multiples of 3
5  for (int i = 1; i <= 15; i++)
6  {
7      if (i % 3 == 0)
8      {
9          continue;              // jump to the update i++
10     }
11     Console.Write($"{i} ");
12 }
13 Console.WriteLine();
14
15 // Part 2: read at most 10 numbers; 0 stops early, negatives are skipped
16 int sum = 0;
```

```
int used = 0;
for (int k = 1; k <= 10; k++)
{
    Console.Write($"Number {k} (0 = stop): ");
    int x = int.Parse(Console.ReadLine());
    if (x == 0)
    {
        break;                  // leave the loop immediately
    }
    if (x < 0)
    {
        Console.WriteLine("  negative -> skipped");
        continue;              // go on with the next k
    }
    sum += x;
    used++;
}
Console.WriteLine($"Sum of {used} positive numbers = {sum}");
```

Sample run

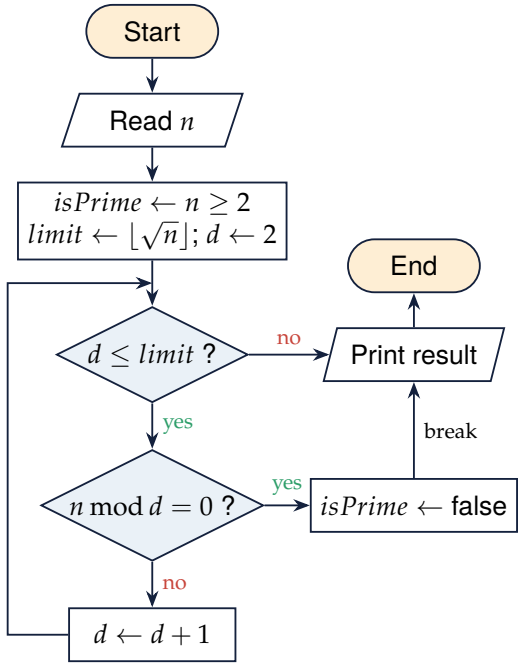
```
1 2 4 5 7 8 10 11 13 14
Number 1 (0 = stop): 8
Number 2 (0 = stop): -3
  negative -> skipped
Number 3 (0 = stop): 12
Number 4 (0 = stop): 5
Number 5 (0 = stop): 0
Sum of 3 positive numbers = 25
```

k	x	sum	$used$	action
1	8	8	1	added
2	-3	8	1	`continue` (skipped)
3	12	20	2	added
4	5	25	3	added
5	0	25	3	`break`

The first line comes from Part 1: `continue` skips $3, 6, 9, 12, 15$. In Part 2 the loop allows at most ten numbers, but the user stops after five (trace table above, right). Use `break` and `continue` sparingly: every extra exit makes a loop harder to trace. Next week we will express "read until 0" more naturally with `while`.

7.1 Prime test by trial division

A number $n \geq 2$ is **prime** if its only divisors are 1 and n. Divisors come in pairs $d \cdot \frac{n}{d}$, and if both members of a pair were larger than $\sqrt{n}$ their product would exceed n. Hence it is enough to try $d = 2, 3, \ldots, \lfloor \sqrt{n} \rfloor$, and the search can stop at the first divisor found.



Algorithm 3 – Prime test

```
1  START
2  READ n
3  isPrime ← (n ≥ 2)
4  FOR d ← 2 TO ⌊√n⌋
5      IF n mod d = 0 THEN
6          isPrime ← false
7          leave the loop (break)
8      END IF
9  END FOR
10 PRINT isPrime
11 END
```

Listing 14: Example 8 – prime test (`06_08_PrimeTest`)

```
// MCMP113 · Week 6 · Example 8 — Prime test by trial division
// Demonstrates a search loop that stops with break, testing divisors only up to √n.

Console.Write("n = ");
int n = int.Parse(Console.ReadLine());

bool isPrime = n >= 2;          // 0, 1, negatives: not prime
int limit = (int)Math.Sqrt(n);  // largest divisor to test
int checks = 0;                 // divisions tried

for (int d = 2; d <= limit; d++)
{
    checks++;
    if (n % d == 0)
    {
        Console.WriteLine($"{n} = {d} x {n / d}");
        isPrime = false;
        break;                  // one divisor is enough
    }
}

if (isPrime)
{
    Console.WriteLine($"{n} is prime");
}
else
{
    Console.WriteLine($"{n} is not prime");
}
Console.WriteLine($"Divisions tried: {checks}");
```

Sample run

```
n = 97
97 is prime
Divisions tried: 8
```

Sample run

```
n = 91
91 = 7 x 13
91 is not prime
Divisions tried: 6
```

Sample run

```
n = 1000003
1000003 is prime
Divisions tried: 999
```

For $n = 91$: $limit = \lfloor\sqrt{91}\rfloor = 9$, and $91 \bmod d$ is $1, 1, 3, 1, 1, 0$ for $d = 2, \ldots, 7$, so the sixth division finds $91 = 7 \cdot 13$. For $n = 1\,000\,003$ (prime) only $d = 2, \ldots, 1000$ are tried: 999 divisions instead of about a million. The initial value `isPrime = n >= 2` handles 0, 1 and negative numbers without an extra `if`; for them the loop does not run at all:

Sample run

```
n = 1
1 is not prime
Divisions tried: 0
```

7.2 All primes up to N

Putting the prime test inside a counting loop lists all primes up to N. Here the condition `d * d <= n` replaces the square root; it means the same, $d \le \sqrt{n}$ (for n above about $2.1 \cdot 10^9$ the product `d * d` would overflow `int`, which is irrelevant here).

Listing 15: Example 9 – primes up to N (`06_09_PrimesUpTo`)

```
// MCMP113 · Week 6 · Example 9 — All primes up to N (nested loops + break)
// Demonstrates an inner search loop inside an outer counting loop.

Console.Write("Primes up to: ");
int max = int.Parse(Console.ReadLine());

int found = 0;
for (int n = 2; n <= max; n++)          // outer: candidates
{
    bool isPrime = true;
    for (int d = 2; d * d <= n; d++)    // inner: divisors
    {
        if (n % d == 0)
        {
            isPrime = false;
            break;                      // inner loop only
        }
    }
    if (isPrime)
    {
        found++;
        Console.Write($"{n,4}");
        if (found % 10 == 0)
        {
            Console.WriteLine();        // 10 per line
        }
    }
}
Console.WriteLine();
Console.WriteLine($"{found} primes found.");
```

Sample run

```
Primes up to: 100
   2   3   5   7  11  13  17  19  23  29
  31  37  41  43  47  53  59  61  67  71
  73  79  83  89  97
25 primes found.
```

The `break` leaves only the *inner* loop; the outer loop continues with the next candidate. `found % 10 == 0` starts a new line after every tenth prime.

7.3 FizzBuzz

For $i = 1, \ldots, n$ print `FizzBuzz` if i is a multiple of 15, `Fizz` for other multiples of 3, `Buzz` for other multiples of 5, and the number otherwise.

Listing 16: Example 11 – FizzBuzz (`06_11_FizzBuzz`)

```
// MCMP113 · Week 6 · Example 11 — FizzBuzz
// Demonstrates a for loop combined with an else-if ladder (order of tests matters).

Console.Write("Up to: ");
int n = int.Parse(Console.ReadLine());

for (int i = 1; i <= n; i++)
{
    if (i % 15 == 0)        // divisible by 3 AND 5: test first
    {
        Console.WriteLine("FizzBuzz");
    }
    else if (i % 3 == 0)
    {
        Console.WriteLine("Fizz");
    }
    else if (i % 5 == 0)
    {
        Console.WriteLine("Buzz");
    }
    else
    {
        Console.WriteLine(i);
    }
}
```

Sample run

```
Up to: 15
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
```

As in Week 5, the order of an else-if ladder matters: if `i % 3 == 0` were tested first, 15 would print `Fizz` and the `FizzBuzz` branch would never be reached.

Test plan. Choose inputs that reach every branch: 1 (number only), 3 (`Fizz`), 5 (`Buzz`), 15 (`FizzBuzz`). The run with $n = 15$ covers all four; $n = 0$ prints nothing, because the condition `1 <= 0` is false at once.

>_ 8 Common mistakes

All outputs and messages below are real.

1. Off-by-one

Goal: print 1 to 10.

```
for (int i = 1; i < 10; i++)
{
    Console.Write($"{i} ");
}
```

prints `1 2 3 4 5 6 7 8 9`: only nine numbers. Use `i <= 10`. Always check the *first* and the *last* value of the loop variable.

2. Semicolon after the header

```
int count = 0;
for (int i = 1; i <= 5; i++);
{
    count++;
}
Console.WriteLine(count);
```

The body of the loop is the empty statement `;`. The loop runs five times doing nothing; then the block runs once: the program prints `1`, not `5`. The compiler only warns (`CS0642: Possible mistaken empty statement`). Read warnings!

3. Changing the loop variable in the body

```
for (int i = 1; i <= 10; i++)
{
    Console.Write($"{i} ");
    i++;   // second update!
}
```

prints `1 3 5 7 9`. Let the header do the updating; if you need steps of 2, write `i += 2` in the header.

4. Accumulator declared inside the loop

```
for (int i = 1; i <= 3; i++)
{
    int sum = 0;   // reset every pass!
    sum += i;
}
```

`sum` starts again at 0 in every pass and does not exist after the loop. Declare accumulators before the loop.

5. Endless loops

- `for (int i = 1; i <= 10; i--)` counts in the wrong direction. The condition stays true while i goes $1, 0, -1, \ldots$; the loop ends only after 2 147 483 650 passes, when `int` overflows from $-2\,147\,483\,648$ to $+2\,147\,483\,647$.
- `for (;;)` has no condition at all and never ends by itself. Stop a running console program with **Ctrl+C**.
- `for (double x = 0; x != 1.0; x += 0.1)`: 0.1 is not exact in binary (Week 4); after ten steps $x = 0.9999999999999999$, so `x != 1.0` stays true forever. Count with `int`, and compare with `<` or `<=`, not `!=`.

6. Integer division in the average

With `int sum` and `int count`, `sum / count` is an integer division: $10/4 = 2$. Write `(double)sum / count` or `sum / 4.0` (Week 4).

>_ 9 Summary

1. `for (init; condition; update) { body }`: init once, then test → body → update until the test is false; n passes need $n + 1$ tests.

2. `for` is a pre-test, counter-controlled loop; its body may run zero times.
3. Flowchart: process (init), decision (condition), body, process (update), back-edge above the decision.
4. A trace table has one row per pass plus a start row and an exit row.
5. Accumulators start at 0 (sums, counts) or 1 (products) before the loop; averages are computed after it, only if the count is positive.
6. Nested loops: the inner loop runs completely for every outer pass ($n \cdot m$ body runs).
7. `break` leaves the innermost loop; `continue` jumps to the update.
8. Classic bugs: off-by-one, `;` after the header, changing i in the body, wrong direction, `double` counters, silent overflow.

Glossary (English – Turkish)

loop	döngü	nested loop	iç içe döngü
iteration / pass	yineleme / tur	inner / outer loop	iç / dış döngü
loop variable (counter)	döngü değişkeni (sayaç)	accumulator	biriktirici (toplayıcı)
initialization	ilk değer atama	product	çarpım
condition	koşul	factorial	faktöriyel
update (increment / decrement)	güncelleme (artırma / azaltma)	overflow	taşma
loop body	döngü gövdesi	scope	kapsam (etki alanı)
pre-test loop	ön koşullu döngü	off-by-one error	bir eksik/fazla hatası
counter-controlled loop	sayaç kontrollü döngü	infinite loop	sonsuz döngü
trace table	izleme tablosu	prime number	asal sayı
back-edge	geri dönüş oku	divisor	bölen
pattern	desen	compound interest	bileşik faiz

>_ 10 Exercises

For every programming exercise: write the pseudocode or draw the flowchart first, then the C# program, then test it with one normal and one edge-case input. Use only what we have learned so far (no `while`, no arrays). Difficulty: ⋆ basic, ⋆⋆ intermediate, ⋆⋆⋆ challenging.

10.1 In-class exercises (pairs)

A. ⋆ **Odd sum.** Read N and print $1 + 3 + 5 + \ldots$ (all odd numbers up to N). Use a step of 2 in the header.

B. ⋆ **Powers of 2.** Read N and print $2^0, 2^1, \ldots, 2^N$, one per line, using a product accumulator (no `Math.Pow`).

C. ⋆⋆ **Divisors and perfect numbers.** Read n; print all its divisors, their number, and whether n is *perfect* (equal to the sum of its divisors smaller than itself, like $6 = 1 + 2 + 3$).

D. ⋆⋆ **Hollow square.** Read n and print an $n \times n$ square of `*` that is empty inside.

E. ⋆⋆ **Sign counter.** Read N integers and print how many are positive, negative and zero.

F. ⋆⋆ **Floyd's triangle.** Read the number of rows and print `1 / 2 3 / 4 5 6 /` ...: row r contains r consecutive numbers.

10.2 Homework

H1. ⋆ Compute $1^2 + 2^2 + \cdots + n^2$ with a loop and compare it with the formula $\frac{n(n+1)(2n+1)}{6}$.

H2. ⋆ Print a Celsius → Fahrenheit table from $-20\,^\circ\text{C}$ to $40\,^\circ\text{C}$ in steps of 10, using $F = \frac{9}{5}C + 32$ (Week 1 exercise F).

H3. ⋆⋆ Print a *diamond*: the pyramid of Example 6 with n rows, followed by the same pyramid upside down without its widest row ($2n - 1$ rows in total).

H4. ⋆⋆ Read N exam scores and print their average, the highest and the lowest score, and how many are at least 50 (combine Examples 2 and 12).

H5. ⋆⋆⋆ Print all perfect numbers up to 10 000 (see exercise C). Use nested loops.

H6. ⋆ On paper: make a trace table (i, condition, action, x) for the fragment below and give its output.

```
int x = 0;
for (int i = 1; i <= 4; i++)
{
    if (i % 2 == 0)
    {
        continue;
    }
    x += i * i;
}
Console.WriteLine(x);
```

10.3 Extra practice

P1. ⋆ How many times does each body run? (a) `for (int i = 3; i <= 30; i += 3)` (b) `for (int i = 100; i >= 0; i -= 10)` (c) `for (int i = 1; i < 1000; i *= 10)` (d) an outer loop $i = 1..5$ around an inner loop $j = 1..4$ (e) an outer loop $i = 1..6$ around an inner loop $j = 1..i$ (f) `for (int i = 10; i < 5; i++)`.

P2. ⋆ Translate into C# and give the output: **FOR** $i \leftarrow 20$ **TO** 2 **STEP** -3: **PRINT** i **END FOR**.

P3. ⋆⋆ **Find the bug.** The program should print the average 2.5 of $1, 2, 3, 4$. What does it print, and why?

```
int sum = 0;
for (int i = 1; i <= 4; i++)
{
    sum += i;
}
double average = sum / 4;
Console.WriteLine(average);
```

P4. ⋆⋆ Predict the output (`break` in a nested loop):

```
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 3; j++)
    {
        if (j == 2)
        {
            break;
        }
        Console.Write($"{i}{j} ");
    }
}
```

P5. ⋆⋆ **Armstrong numbers.** Find all three-digit numbers equal to the sum of the cubes of their digits (e.g. $153 = 1^3 + 5^3 + 3^3$). Use three nested loops over the digits.

P6. ⋆⋆ **Pythagorean triples.** Print all triples $a < b < c \leq 30$ with $a^2 + b^2 = c^2$ and count them.

P7. ⋆⋆⋆ **Approximating π.** Leibniz's series $\pi = 4\left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots\right)$. Read the number of terms and print the approximation with 6 decimals next to `Math.PI`.

References and further reading

[1] Microsoft, *Iteration statements – for, foreach, do, and while* (C# reference). https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/iteration-statements

[2] Microsoft, *Jump statements – break, continue, return, and goto* (C# reference). https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/jump-statements

[3] Microsoft, *The checked and unchecked statements* (C# reference). https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/checked-and-unchecked

[4] Microsoft, *Integral numeric types* (ranges of `int` and `long`). https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/integral-numeric-types

[5] Microsoft, *Compiler Warning (level 3) CS0642*. https://learn.microsoft.com/en-us/dotnet/csharp/misc/cs0642

[6] Microsoft, *Standard numeric format strings* (`F2`, `N0`). https://learn.microsoft.com/en-us/dotnet/standard/base-types/standard-numeric-format-strings

[7] D. E. Knuth, *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3rd ed., Addison-Wesley, 1997, Section 1.2.8 (Fibonacci numbers).